10. Hafta
SQL Dili

SQL tablolar ve bunların arasındaki ilişkilerle çalışmak için kullanılan bir bilgisayar dilidir. Microsoft Office Access gibi ilişkisel veritabanı programları verilerle çalışmak için SQL kullanır. Çoğu bilgisayar dilinin aksine SQL'in okunması ve anlaşılması yeni kullanan biri için bile zor değildir. Birçok bilgisayar dili ile aynı şekilde SQL de ISO ve ANSI gibi standart kuruluşları tarafından tanınan uluslararası bir standarttır.

Select Cümleciği:

Select cümleleri istediğimiz tablolardan istediğimiz satırları seçmek ve göstermek için kullanılır. Temel kullanımı

SELECT ALANADI_1, ALANADI_2 FROM TABLO_ADI

şeklindedir. Select ifadesinden sonra listelenmek istenen alan adları aralarına virgül konularak sıralanır, from kelimesi eklenerek, from dan sonra hangi tablodaki kayıtların listeleneceği seçilir.

Bütün alanlar seçilmek istenirse alan adları yerine * konulabilir.

<u>Örnek:</u>
Select * from KIMLIKBILGILERI
Yukarıdaki select cümlesi KIMLIKBILGILERI tablosundaki bütün kayıtları listeleyecektir.

Sadece belirli kayıtlar listelenmek istenirse, örneğin sadece adı Ali olan personel listelenmek istenirse WHERE kelimesinin kullanılması gerekmektedir. WHERE kelimesinden sonra kontrol edilecek alan ve değeri belirtilmelidir. Sadece adı ali olan kayıtları listelemek için yazılacak SQL şu şekildedir:

SELECT * FROM KIMLIKBILGILERI WHERE AD="Ali"

Birden çok şart kontrol edilmek istenirse aralarına AND konularak arka arkaya sıralanabilir. Örneğin Adı Ali Soyadı Can olan kayıtları listelemek için;

SELECT * FROM KIMLIKBILGILERI WHERE AD="Ali" AND SOYAD="Can"

cümlesi yazılabilir.

Insert Cümleciği:

Tabloya yeni bir satır eklemek için kullanılır. Örneğin Birimler tablosuna yeni bir kayıt eklemek için yazılması gereken SQL cümlesi aşağıdaki şekilde yazılabilir.

INSERT INTO BIRIMLER VALUES (6,'Yeni Birim')

Insert cümlesi yukarıdaki gibi yazılabileceği gibi daha güvenli bir çalışma için aşağıdaki şekilde yazılması daha doğru olacaktır.

INSERT INTO BIRIMLER (BIRIM_NO,ACIKLAMA) VALUES (6,"Yeni Birim")

Update Cümleciği:

Bir tablo içerisindeki bir alanın değerini değiştirmek için kullanılır. Select cümlesinde olduğu gibi belirli satırları veya tüm satırları güncellemek için kullanılabilir.
Tüm satırları güncellemek için;

UPDATE BIRIMLER SET ACIKLAMA="Teftiş"

şeklinde yazılabilir yukarıdaki cümle BIRIMLER tablosundaki bütün satırların ACIKLAMA alanını Teftiş olarak güncelleyecektir.

Sadece belirli şartlara uyan satırları güncellemek için aşağıdaki şekilde bir SQL cümlesi yazılabilir.

UPDATE BIRIMLER SET ACIKLAMA="Teftiş" WHERE BIRIM=6

Delete Cümleciği:

DELETE cümlesi bir tablo içerisindeki tüm satırları veya belirli satırları silmek için kullanılır. Tüm satırları silmek için;

DELETE FROM ACIKLAMA

Şeklinde yazılabilir. Sadece belirli bir şartı tutan satırları silmek için;

DELETE FROM ACIKLAMA WHERE BIRIM=6
şeklinde yazılabilir.

C# ile Veritabanı İşlemleri:

Ado.Net (Activex Data Object):

Bir çok programlama dilinde arka planda çalışan ADO(Activex Data Object)dir. ADO.NET uygulamalarımızda veri kullanmamızı kolaylaştıran .Net Framework sınıf kütüphanelerindendir.

ADO.NET başka programlama dillerinde de kullanılması için Microsoft tarafından bütün database bağlantı uygulamalarının birbirleriyle entegrasyonunu yaparak nesneler şeklinde paketlenmiş halidir.

ADO.NET içerisindeki sınıflar, altyapının ve veritabanına özgü karmaşık işlemlerin çoğunu yönetir.Böylece bu da yazılımcıların geliştirdikleri programlarda, veri okuma ve yazmada daha az kod kullanmaları yeterli olacaktır.

ADO.NET bir dil değil sadece yaygın işlevselliği sağlayan bir sınıf derlemesidir.

C# projelerinde bir veritabanı bağlantısı oluşturmak ve veritabanı üzerinde çalışmak için ilk olarak ilgili namespace in projeye dahil edilmesi gerekmektedir.

Access üzerinde çalışmak için eklenmesi gereken namespace aşağıda görülmektedir. OleDb(object linking and embedding **database)**

using System.Data.OleDb;

OleDbConnection Nesnesi:

Bir veri tabanı bağlantısı oluşturmak için kullanılan nesnedir. Veri tabanı bağlantısı oluşturulmadan önce OleDbConnection nesnesinden bir örnek oluşturularak bu nesnenin ConnectionString özelliği ayarlanmalıdır. Daha sonra open metodu kullanılarak bağlantı açılır ve işlemler yapıldıktan sonra Close metodu kullanılarak bağlantı kapatılır.

ConnectionString özelliğinde bağlanmak istediğimiz veritabanımızın yolunu belirtmemiz gerekmektedir.

Örnek:

```
OleDbConnection Baglanti = new OleDbConnection();
Baglanti.ConnectionString = "Provider=Microsoft.Jet.OLEDB.4.0;Data
Source='d:\\Veritabanı1.mdb';Persist Security Info=True";
Baglanti.Open();
Baglanti.Close();
```

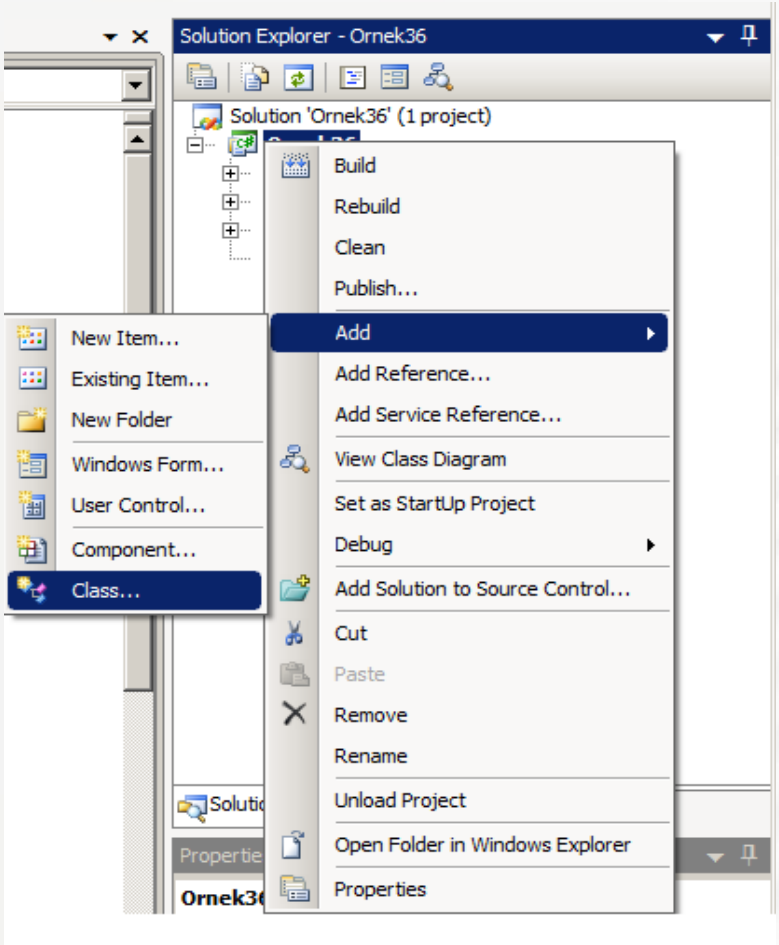
Yukarıdaki kod örneğinde ConnectionString içerisine bağlantı sağlayıcının provider'i dosya yolu ve bağlantı ile ilgili ek tanımlar bulunmaktadır. Veritabanı dosyamızı da D dizinin altına kopyalamamız gerekmektedir.

Veritabanı işlemleri için örnek bir class oluşturulması:

Veritabanı işlemlerinin daha düzenli ve fonksiyonel olmasını C# projemizde bu işlemleri bir class altında toplayarak sağlayabiliriz.

Projeye yeni bir class eklemek için proje üzerinde sağ tıklayarak Class seçeneğini seçilir.

Bu şekilde projemizde ihtiyacımız olan her yerde ilgili classımızdan bir nesne örneği yaratarak kullanabiliriz.


Solution Explorer - Ornek36
Solution 'Ornek36' (1 project)
Build
Rebuild
Clean
Publish...
Add
Add Reference...
Add Service Reference...
View Class Diagram
Set as StartUp Project
Debug
Add Solution to Source Control...
Cut
Paste
Remove
Rename
Unload Project
Open Folder in Windows Explorer
Properties
New Item...
Existing Item...
New Folder
Windows Form...
User Control...
Component...
Class...

Oluşturulan class dosyamızın otomatik olarak üretilen kodu aşağıdaki şekilde olacaktır. Classımıza isim olarak VeritabaniIslemleri verdik.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;


namespace Ornek36
{
    class VeritabaniIslemleri
    {

    }
}
```

İlk olarak classımıza **using System.Data.OleDb;** satırını ve **using System.Data;** satırını ekleyerek veritabanı işlemleri için ilgili kütüphaneleri classımıza dahil etmeliyiz. Classımıza Veritabanından veri çekmek için

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Data.OleDb;
using System.Data;

namespace Ornek36
{
    class VeritabaniIslemleri
    {
        public DataTable VeriGetir(string sorgum)
        {
            string connStr = "Provider=Microsoft.Jet.OLEDB.4.0;Data
Source='d:\\Veritabanı1.mdb';Persist Security Info=True";
            OleDbConnection conn = new OleDbConnection();
            conn.ConnectionString = connStr;
            DataTable dt = new DataTable();
            OleDbDataAdapter dadp = new OleDbDataAdapter(sorgum, conn);
            conn.Open();
            dadp.Fill(dt);
            conn.Close();
            return dt;
        }
    }
}
```

DataTable nesnesi System.Data; namespace i içerisinde bulunamaktadır. Ve veritabanı üzerinde çalıştırılan selecet cümleciklerinin sonuçlarını tutmak için kullanılır. OleDbDataAdapter nesnesi mevcut bir Veritabanı bağlantısı üzerinde istenilen bir select sorgusunu çalıştırırarak sonucunu Fill metodu ile parametre olarak verilen bir DataTable doldurmaktadır. Yukarıdaki yöntemde dönüş türü olarak DataTable seçildiğinden return komutundan sonra bu dataTable döndürülmektedir.

Örnek 36: Bu örnekte yukarıda tasarımı yapılan veritabanı üzerinde oluşturulan class kullanılarak kayıt çekme işlemi uygulanmaktadır.

Örnek için hazırlanan formumuzun tasarım görünümü:

İlk olarak projemiz içerisinde lazım olan değişkenleri ve nesneleri classımızın başında tanımlıyoruz.

```
public int kayitNo = 0;
DataTable dt = new DataTable();
```

Veritabanından gelen verileri saklamak için bir data table, ekranda gösterilecek olan kayıtların sırasını takip etmek için kayitNo değişkenimiz bulunmaktadır.

Verileri Getir Butonu içerisine yazılan kod:

```
VeritabaniIslemleri vt = new VeritabaniIslemleri();

dt = vt.VeriGetir("SELECT * FROM KIMLIKBILGILERI");
textBox5.Text = dt.Rows.Count.ToString();
```

Bu bölümde ilk olarak hazırladığımız classımızdan vt isimli bir nesne oluşturuyoruz ve vt nesnemiz içerisindeki VeriGetir yöntemine sorgumuzu göndererek sonuçları dataTable'mıza aktarıyoruz. Toplam kayıt sayısı alanına datatable nesnesinin kayıt sayısını yazdırıyoruz.

Önceki kayıt butonuna yazılan kod:

```
if (kayitNo == 0)
{
    MessageBox.Show("İlk Kayıttasınız Geri Gidemezsiniz");
}
else
{
    kayitNo--;
    textBox1.Text = dt.Rows[kayitNo]["TC_KIMLIK_NO"].ToString();
    textBox2.Text = dt.Rows[kayitNo]["AD"].ToString();
    textBox3.Text = dt.Rows[kayitNo]["SOYAD"].ToString();
    textBox4.Text = dt.Rows[kayitNo]["TELEFON"].ToString();
}
```

İf şartımız ile, ilk kayıtta ise programın hata vermesini sağlıyoruz. Değil ise bir önceki kayda konumlanmak için kayitno nun değerini bir azaltıyoruz. Ve sırası ile metin kutularımıza ilgili kayıttaki değerleri yazdırıyoruz.

```
dt.Rows[kayitNo]["TC_KIMLIK_NO"].ToString();
```

satırında rows özelliği data table miz içerisinde bulunan satırlara ulaşmamızı sağlamaktadır. Bu özellik ile istenilen satırdaki istenilen alana ulaşılabilir. Bu özellik bir iki boyutlu dizi gibi düşünülebilir.

Sonraki kayıt butonuna yazılan kod:

```
if (kayitNo == dt.Rows.Count-1)
{
    MessageBox.Show("Son Kayıttasınız İleri Gidemezsiniz");
}
else
{
    kayitNo++;
    textBox1.Text = dt.Rows[kayitNo]["TC_KIMLIK_NO"].ToString();
    textBox2.Text = dt.Rows[kayitNo]["AD"].ToString();
    textBox3.Text = dt.Rows[kayitNo]["SOYAD"].ToString();
    textBox4.Text = dt.Rows[kayitNo]["TELEFON"].ToString();
}
```

Bu buton içerisine yazılan kodun bir önceki kod parçacığından farkı datatable'in sonunda olup olmadığını kontrol etmesi ve kaydı göstermeden önce gösterilecek kayit numarasını bir artırmasıdır.

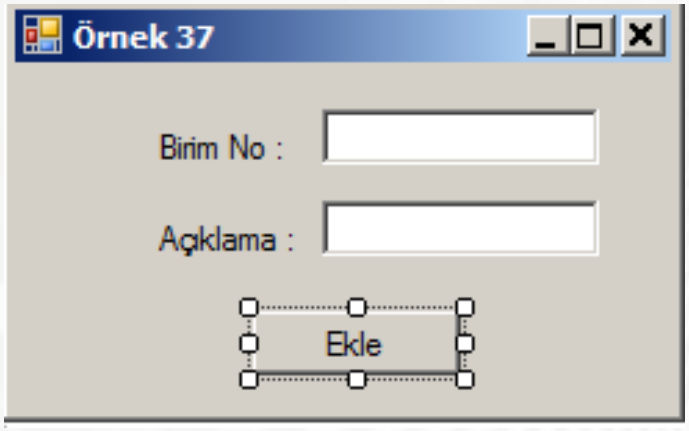
VeriTabanı işlemleri classına VeriEkleSilGuncelle Yönteminin yazılması:

Veritabanımız üzerinde ekleme, silme ve ya güncelleme işlemlerini yapabilmemiz için classımız içerisine aşağıdaki gibi bir yöntem ekleyebiliriz.

```
        public int VeriEkleSilGuncelle(string sorgum)
        {
            string connStr = "Provider=Microsoft.Jet.OLEDB.4.0;Data
Source='d:\\Veritabanı1.mdb';Persist Security Info=True";

            OleDbConnection conn = new OleDbConnection();
            conn.ConnectionString = connStr;

            OleDbCommand cmd = new OleDbCommand();

            cmd.CommandText = sorgum;
            cmd.Connection = conn;

            conn.Open();
            int sonuc = cmd.ExecuteNonQuery();
            conn.Close();

            return sonuc;
        }
```

Bu yöntemimizin dönüş değeri int verilmiştir bu yöntemde dönüş değeri işlemden etkilenen satır sayısı olacaktır. Veri eklemek silmek ve ya güncellemek için OleDbCommand nesnesi kullanılabilir. bu nesnenin CommandText özelliğine yazılan sql deyimi ExecuteNonQuery() metodu ile çalıştırılır.

Ekle butonu içerisine yazılan kod:



- Veritabanı class'ımızdan bir örnek oluşturarak onun VeriEkleSilGuncelle Metodunu kullanıyoruz.

```
VeritabaniIslemleri vt = new VeritabaniIslemleri();
vt.VeriEkleSilGuncelle("INSERT INTO  BIRIMLER (BIRIM_NO,ACIKLAMA) VALUES
("+textBox1.Text+",'"+textBox2.Text+"')");
```

- Yukarıdaki işlemde veri tabanı ile bir bağlantı oluşturulmuştur. Insert into komutuyla veritabanına kayıt eklenmektedir. Tablo adi veritabanındaki oluşturulmuş tablonun adıdır ve içindeki değerlerde alan adlarıdır. Values ile bzizim istediğimiz değerleri bu alan adlarına sırasıyla göndermekteyiz. komut.Connection.Open(); ile bağlantıyı açarız ve komut.ExecuteNonQuery(); ile yazdığımız kodların çalışmasını sağlarız ve close ile bağlantıyı kapatırız. Komutlar hep böyle olduğu için bundan sonraki komutlarda sadece insert into ile başlayan kısımlar verilecektir.
- }

- Burada var olan bir kaydı silmek için aşağıdaki kod yazılır. Aşağıdaki kod İnsert into kodunun yerine yazılması gereklidir. Where komutuyla ad alanında ahmet ismine eşit olan tüm kayıtları buluyoruz ve delete komutuyla bunları siliyoruz.
- DELETE FROM tabloadi WHERE ad='ahmet';
- Tabloda var olan bir kaydı değiştirmek istiyorsak update komutu kullanılır.
- UPDATE tabloadi SET ad = 'Bilişim', soyad='enstitüsü' WHERE ad = 'ahmet'
- Yukarıdaki kod tabloadi ismindeki tablodan ad kısmı ahmet ismine eşit olan kayıtlarda ad alanını Bilişim, soyad alanını da enstitüsü yap anlamına gelmektedir.

- Bu SQL cümleciklerinde kullanılabilecek bazı diğer komutlar aşağıda verilmiştir.
- **Between**
- Select * from tabloadi where maas between 10000 and 1000000
- Maas alanı olduğunu varsayarsak tabloda yukarıdaki kodta 10000 ile1000000 arasında maaşa sahip olan tüm kayıtları göster anlamına gelmektedir.
- **Like**
- Select * from tabloadi where ad like ' %y% '
- Tabloda ad alanında y karakteri içeren tüm kayıtları göster anlamına gelmektedir.

- **ORDER BY**
- Select * from tabloadi order by ad ASC
- Tablodaki tüm kayıtları ad alanına göre küçükten büyüğe sırala anlamına gelmektedir. ASC yerine DESC yazılırsa büyükten küçüğe sıralama yapar.
- **Count**
- Select count(ad) from tabloadi where ad='ahmet'
- Tabloda ad alanında ahmet ismine eşit kaç tane kayıt varsa sayısal olarak değerini dönderir.
- **Visual C#.NET'te SQL Server** bölümünü tamamladınız.
- Bu bölümde;
- C#.NET ile veri tabanına sihirbaz kullanarak ve kod kullanarak nasıl bağlanıldığını öğrendiniz ve veritabanıyla işlem yapabilme becerisi kazandınız.